

Group 15 — Digital Twin

Data Pipeline — API Reference v1.0

TU Berlin · CPS · June 2026

1 Overview.....	3
2 Configuration (run_pipeline.py).....	3
3 EnvironmentPipeline.....	3
EnvironmentPipeline(broker, port, weather_fallback, window_seconds, connect)	3
start().....	4
get_data(field_name=None)	4
get_average(field_name)	4
4 WeatherFallback.....	5
WeatherFallback(latitude, longitude).....	5
get_temperature()	5
get_humidity()	5
get_pressure().....	5
5 CSV Log (sensor_log.csv).....	5
File format	6
Reading the CSV — read_sensor_log.py	6
CSV helper reference.....	6
6 Fallback Chain.....	7
7 Quick Demo.....	7

1 Overview

Two files are all you need:

Parameter / Field	Type	Description
<code>pipeline_with_fallback.py</code>	Core module	Contains all classes. Import this to use the pipeline in your own code.
<code>run_pipeline.py</code>	Entry point	Run this file to start the pipeline. Edit <code>BROKER</code> , <code>PORT</code> , and <code>WINDOW_SECONDS</code> here.

Start the pipeline:

```
|python run_pipeline.py
```

The pipeline connects to the MQTT broker, subscribes to sensor topics (P03, P01, P07), validates every reading, runs a fallback if a sensor fails, and logs trusted data to `sensor_log.csv`.

2 Configuration (`run_pipeline.py`)

Edit these constants at the top of `run_pipeline.py`:

Parameter / Field	Type	Description
<code>BROKER</code>	str	MQTT broker hostname. Default: "broker.hivemq.com". Confirm with P04.
<code>PORT</code>	int	MQTT port. Default: 1883.
<code>WINDOW_SECONDS</code>	float	Rolling window for <code>get_average()</code> . Default: 300 (5 min).
<code>INITIAL_LATITUDE</code>	float	Starting latitude for weather fallback. Updated automatically from P07. Default: 52.52 (Berlin).
<code>INITIAL_LONGITUDE</code>	float	Starting longitude. Default: 13.405 (Berlin).

3 EnvironmentPipeline

Main class. Handles MQTT, validation, fallback chain, and CSV logging.

EnvironmentPipeline(`broker`, `port`, `weather_fallback`, `window_seconds`, `connect`)

Parameter / Field	Type	Description
<code>broker</code>	str	MQTT broker hostname. Example: "broker.hivemq.com"
<code>port</code>	int	MQTT port. Typically 1883.
<code>weather_fallback</code>	WeatherFallback	A WeatherFallback instance. Used when P03 sensor fails.
<code>window_seconds</code>	float	Rolling window for averaging buffer. Default: 300.
<code>connect</code>	bool	Set False in unit tests to skip broker connection. Default: True.

start()

Starts the MQTT loop. Blocks until Ctrl+C.

Returns	Description
None	Runs indefinitely.

```
|pipeline.start() # blocks here - press Ctrl+C to stop
```

get_data(field_name=None)

Get the latest trusted value for one field, or all fields at once.

Parameter / Field	Type	Description
field_name	str None	Name of the field. Pass None (or omit) to get all fields. Example: "temperature_c"

Returns	Description
any dict None	Latest value for that field, or a dict of all fields. None if no valid reading received yet.

```
# Single field
temp = pipeline.get_data("temperature_c")
print(temp) # e.g. 21.3

# All fields
snapshot = pipeline.get_data()
# {'temperature_c': 21.3, 'humidity_rel': 58.1, 'calibrated': 0.61, ...}
```

Available field names:

Parameter / Field	Type	Description
temperature_c	float	Air temperature in °C (from P03, fallback: OpenMeteo)
humidity_rel	float	Relative humidity in % (from P03, fallback: OpenMeteo)
pressure_hpa	float	Atmospheric pressure in hPa (from P03, fallback: OpenMeteo)
light_lux	float	Ambient light in lux (from P03, no fallback)
calibrated	float	Soil moisture 0.0–1.0 (from P01, fallback: last known)
raw_adc	int	Raw ADC value from soil sensor (from P01, no fallback)

get_average(field_name)

Rolling average of a numeric field over the last window_seconds seconds.

Parameter / Field	Type	Description
field_name	str	Name of a numeric field. Example: "temperature_c"

Returns	Description
float None	Rolling average, or None if no data in window.

```
avg = pipeline.get_average("temperature_c")
print(avg)      # e.g. 21.7
```

4 WeatherFallback

Fetches current weather from OpenMeteo. Used automatically by the fallback chain — but can also be used standalone.

WeatherFallback(latitude, longitude)

Parameter / Field	Type	Description
latitude	float None	Initial latitude. Set None to wait for P07 to provide coordinates.
longitude	float None	Initial longitude.

get_temperature()

Returns	Description
float None	Current temperature in °C at the stored coordinates, or None on failure.

```
from pipeline_with_fallback import WeatherFallback

wb = WeatherFallback(latitude=52.52, longitude=13.405)
print(wb.get_temperature())    # e.g. 18.4
print(wb.get_humidity())       # e.g. 62.0
print(wb.get_pressure())       # e.g. 1013.2
```

get_humidity()

Returns	Description
float None	Current relative humidity in % at the stored coordinates, or None on failure.

get_pressure()

Returns	Description
float None	Current surface pressure in hPa, or None on failure.

5 CSV Log (sensor_log.csv)

Every trusted reading is written to sensor_log.csv in the same folder as pipeline_with_fallback.py. The file resets automatically after 8 hours.

File format

Parameter / Field	Type	Description
timestamp_unix	float	Unix timestamp (seconds since 1970). Useful for time arithmetic.
timestamp_utc	str	UTC time in ISO 8601. Example: "2026-06-26T17:04:31Z"
field	str	Field name — one of: temperature_c, humidity_rel, pressure_hpa, light_lux, calibrated, raw_adc.
value	float	The trusted sensor value at that time.

Example rows:

```
timestamp_unix,timestamp_utc,field,value
1750954971.2,2026-06-26T17:04:31Z,temperature_c,21.3
1750954971.3,2026-06-26T17:04:31Z,humidity_rel,58.1
1750954976.1,2026-06-26T17:04:36Z,calibrated,0.612
```

Note: each field is a separate row — sensors publish at different times.

Reading the CSV — read_sensor_log.py

Use the helper functions in read_sensor_log.py. No extra libraries needed.

```
from read_sensor_log import load_log, get_by_time, get_field, get_values, summary

# 1. Load everything
rows = load_log()

# 2. Filter to a time window
window = get_by_time(rows, "2026-06-26T17:00:00Z", "2026-06-26T17:30:00Z")

# 3. Get rows for one field
temps = get_field(window, "temperature_c")

# 4. Get just the numbers
values = get_values(window, "temperature_c")
print(min(values), max(values))

# 5. Quick stats (count, mean, min, max, std dev)
summary(rows, "calibrated")
```

CSV helper reference

Parameter / Field	Type	Description
load_log()	list[dict]	Load entire CSV. Returns list of {timestamp_unix, timestamp_utc, field, value}.
get_by_time(rows, start, end)	list[dict]	Filter by UTC time window. Format: "2026-06-26T17:00:00Z"
get_field(rows, field_name)	list[dict]	Keep only rows for one field.
get_values(rows, field_name)	list[float]	Return just the numeric values as a plain list.

Parameter / Field	Type	Description
summary(rows, field_name)	None	Print count, mean, min, max, std dev to terminal.

6 Fallback Chain

Every numeric field goes through three levels on each message. Stops at the first success.

Parameter / Field	Type	Description
Plan A – Sensor	Always first	Accept the value if sensor status is clean and value is in valid range.
Plan B – Fallback	If A fails	temperature_c / humidity_rel / pressure_hpa → OpenMeteo API. calibrated → last trusted reading (max 5 consecutive misses).
Plan C – Escalation	If B fails	Prints [ERROR] to terminal. No value stored.

Terminal output:

```
[P03] status=ok temperature_c=21.3 humidity_rel=58.1 pressure_hpa=1012.4
light_lux=32000
[WARNING] temperature_c: sensor status='sensor_error', using OpenMeteo fallback = 18.4
[ERROR] [P03] temperature_c unavailable from sensor and all fallbacks
```

7 Quick Demo

```
# Terminal 1
python run_pipeline.py

# Terminal 2 (for testing without real sensors)
python environment_simulator.py
```

Once pipeline is running, query data from the CSV in any separate script:

```
from read_sensor_log import load_log, get_by_time, summary

rows = load_log()
window = get_by_time(rows, "2026-06-26T17:00:00Z", "2026-06-26T18:00:00Z")
summary(window, "temperature_c")
summary(window, "calibrated")
```

Or use WeatherFallback standalone:

```
from pipeline_with_fallback import WeatherFallback
wb = WeatherFallback(latitude=52.52, longitude=13.405)
print(wb.get_temperature(), wb.get_humidity(), wb.get_pressure())
```